Transformers and Large Language Models

DSC 340 Applied Machine Learning

Special Topics

Friday, May 2, 2025

In today's session, we'll explore how **GPT** works and why **transformers** are at the heart of modern AI systems.

This session will build on what we've already learned about **language models** and **neural networks**, diving deeper into the practical mechanics behind systems like ChatGPT.

By the end of this session, you'll understand the key ideas of **tokens**, **embeddings**, and the revolutionary concept of **attention** that made these models so powerful.

GPT

- **Generative**
  - Generate coherent, contextually appropriate responses
- **Pretrained**
  - Trained on massive datasets to predict the next word effectively
- **Transformer**
  - Uses transformer architecture, effective at long-range dependencies

Today, we're focusing on **GPT**, which stands for **Generative Pretrained Transformer**. Each part of the name tells us something important about how these models work.

Generative means GPT creates output—whether it's completing a sentence, answering a question, or generating text from scratch.

Pretrained refers to the extensive initial training on massive amounts of data, which equips the model with broad knowledge about language.

Finally, **Transformer** is the specific architecture that powers GPT, allowing it to handle complex relationships in language over long contexts effectively. This definition will guide us as we explore why GPT and transformers are so revolutionary.

Why Study GPT and Transformers?

- Practical: Powering chatbots, translation, text-to-image, and more
- Theoretical: A leap from traditional neural networks
- Cultural: Ethical, ecological, and societal impacts

Why are we focusing on **GPT** and **transformers** today?

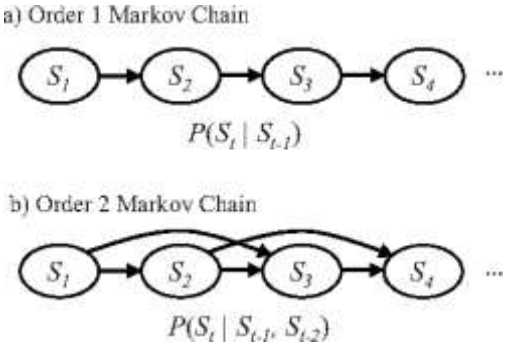
First, they're incredibly **practical**, powering applications like **chatbots**, **language translation**, and even **text-to-image** generation.

Second, from a **theoretical** standpoint, transformers represent a leap beyond earlier methods like **RNNs** and **LSTMs**, solving key problems with long-term dependencies.

Finally, these models are shaping **culture** and **ethics** as we debate their societal and ecological impacts. By understanding how they work, you'll gain a clearer view of their capabilities and challenges.

Language Models Recap

- Goal: Predict next word (token)
  "The dog chased the _____"
- Simple language models
  - **Bag of words**
  - First-order **Markov chain**
  - Second-order Markov chain
- Context matters:
  Long-range dependencies



Language models, as we discussed in a previous session, aim to **predict the next word** in a sequence. This is the foundation of many AI tasks, from **autocomplete** to **machine translation**.

Early methods, like **Markov chains**, made predictions based on a limited window of context. For example, if we start with "The dog chased the…," the model might predict the next word based only on the previous 1-3 words.

While this works for simple patterns, it struggles with **long-range dependencies** and nuanced meaning. This example introduces a common challenge in language modeling, which we'll revisit when discussing transformers.

Language Models → Probability Distribution

- In machine learning, language models are based on probabilities
  - Recall: Distribution learned data (corpus)
- Example: "The dog chased the _____"
  - **Stop and Think: What is your prediction?**
- Possible predictions (completely made up):

```
Ball:        30%
Squirrel:    25%
Mailman:     20%
Car:         20%
Neighbor:     3%
```

Language models in machine learning rely on **probabilities** to predict the next word in a sentence. These probabilities aren't random—they're learned from analyzing large datasets, or **corpora**, during training. Let's consider an example: "The dog chased the _____." Stop for a moment and think—what's your prediction?

Based on the patterns learned from the data, the model might generate a probability distribution like this: "ball" at 30%, "squirrel" at 25%, "mailman" at 20%, "car" at 20%, and "neighbor" at just 3%. These are **made-up values**, but they illustrate how the model quantifies its guesses.

Early methods, like **Markov chains**, could make similar predictions but struggled with using enough context, often leading to less accurate results. This limitation sets the stage for why transformers revolutionized language mode

Tokens and Tokenization

- Breaking text into **tokens** (e.g., words or **subwords**)
- Tokens sit between **letters** and **words** in granularity
- Example: “Out walking!” → [“out”, “walk”, “ing”, “!”]
- Smaller vocabulary: GPT-3 uses ~50,000 tokens vs. ~1M words
- Tokens often relate to **morphemes**, encoding meaning

Tokens are the **units of text** that language models process, and they fall between letters and words in terms of granularity. For example, “Out walking!” could be tokenized into four pieces: “out,” “walk,” “ing,” and “!” Tokens are designed to strike a balance: more granular than words, but less granular than letters. This is key because tracking every word in English, with its vast vocabulary of over 1 million words, would be inefficient. Instead, models like GPT-3 use a smaller **vocabulary of around 50,000 tokens**.

Tokens often correspond to **morphemes**, the smallest meaningful units in language. This means they can encode meaning more effectively than letters, while still handling rare or novel words by breaking them into smaller components.

Tokens and Model Behavior

- Tokens drive model predictions, not words
- Counting tokens ≠ Counting words
  - This is why even models like GPT-4o struggle with precise counts!
  - Example: Asking for exactly 10 words may fail
- Context window: Models "remember" a limited number of tokens

Here's where things get tricky: language models like GPT process text **in tokens, not words**. This can lead to unexpected behavior. For example, if you ask a model to generate exactly 10 words, it might struggle because it's internally working with tokens, which don't always map directly to words. A single word might break into multiple tokens, or a token might represent multiple words in certain cases.

Clarification in response to a question from one of you:

In linguistics, a **phoneme** is the smallest unit of sound in spoken language. For example, the /k/ sound in *cat* is a phoneme. Phonemes distinguish one word from another. A **morpheme**, on the other hand, is the smallest unit of meaning. For example, the prefix *un-* in *unlock* carries the meaning of negation. A single word can have multiple morphemes, such as *un-lock-able*, which contains three morphemes.

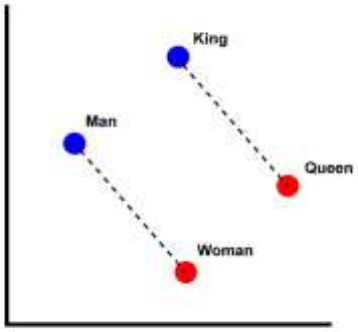
In natural language processing, **tokens** are the building blocks models use to make predictions. Tokens are not the same as words; they may represent words, parts of words, or even individual characters. This distinction is important because counting **tokens** does not equal counting **words**. For instance, the word *unlockable* might be split into three tokens: *un*, *lock* and *able*. This tokenization process can lead to

challenges in tasks like asking models to generate exactly ten words, as their predictions are based on tokens, not whole words.

Tokenization often *tries* to capture **morphemes** because they are meaningful units, but it does not always do so perfectly. This is because tokenization relies on simple rules and patterns, not a deep understanding of meaning.

From Tokens to Embeddings

- Tokens become **vectors** in a high-dimensional space
- Embeddings encode **meaning** and **relationships**
- Example:
  "king" – "man" + "woman" ≈ "queen"
- Word embeddings cluster similar words together



Once we've tokenized text, the next step is turning those tokens into **vectors**—mathematical representations in a high-dimensional space. These vectors are known as **embeddings**, and they're powerful because they allow models to capture **relationships** between words. For example, in a well-trained embedding space, the vector operation "king – man + woman" results in a vector close to "queen." This reflects how embeddings encode **meaning**.

Another key feature of embeddings is that similar words—like "happy" and "joyful"—tend to cluster together in this space. This clustering allows models to generalize meaning effectively and understand **semantic relationships** between words.

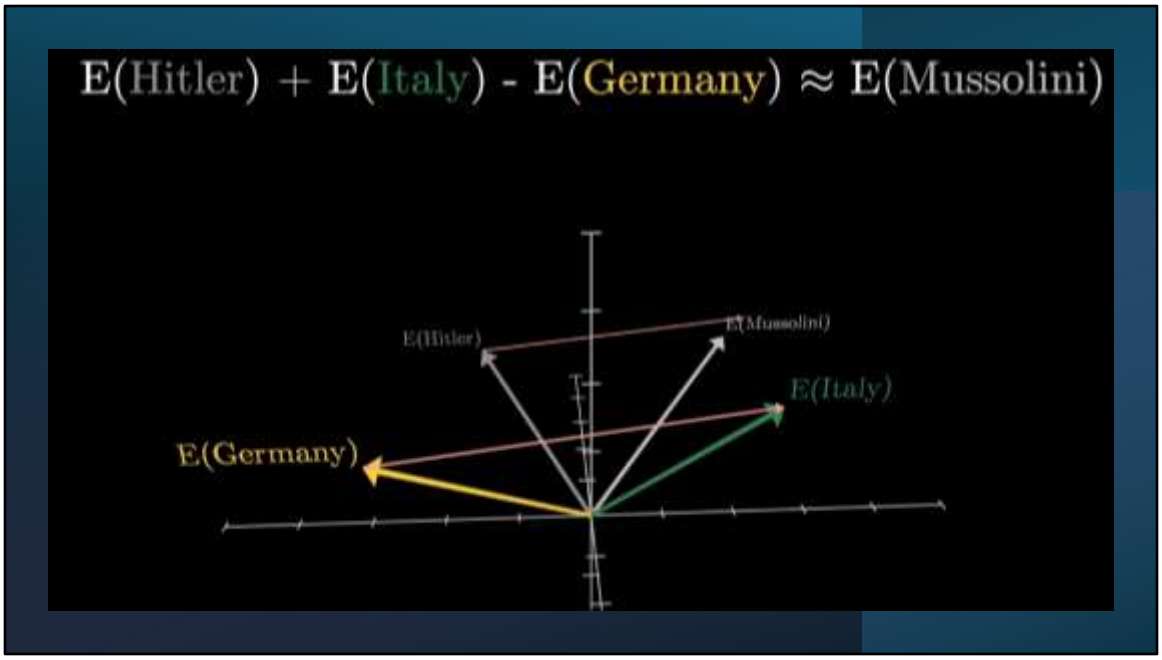
Understanding Word Embeddings

- Words with similar meanings are **closer** in the embedding space
- Contextual embeddings: Meaning changes with usage (e.g., “bank”)
- Pretrained embeddings save time and improve accuracy
- Applications: Search engines, translation, chatbots

Let’s dive deeper into **word embeddings**. In this high-dimensional space, words with similar meanings or usage—like “happy” and “joyful”—are **closer together**. This property enables models to generalize across related terms, making them more effective in tasks like translation or question-answering.

Some embeddings are **contextual**, meaning they adjust based on how a word is used. For instance, “bank” could refer to a riverbank or a financial institution, and a good embedding will reflect the context to disambiguate its meaning.

Modern embeddings are often **pretrained** on large datasets, saving time and improving accuracy when fine-tuned for specific tasks. They power applications like search engines, chatbots, and machine translation, making embeddings a cornerstone of AI systems.



Source: From the assigned reading: https://www.3blue1brown.com/lessons/gpt

Note: In the embedding data for OpenAI's GPT3, this example analogy (Hitler : Germany :: Mussolini : Italy) actually works better than the classic example (king : man :: queen : woman).

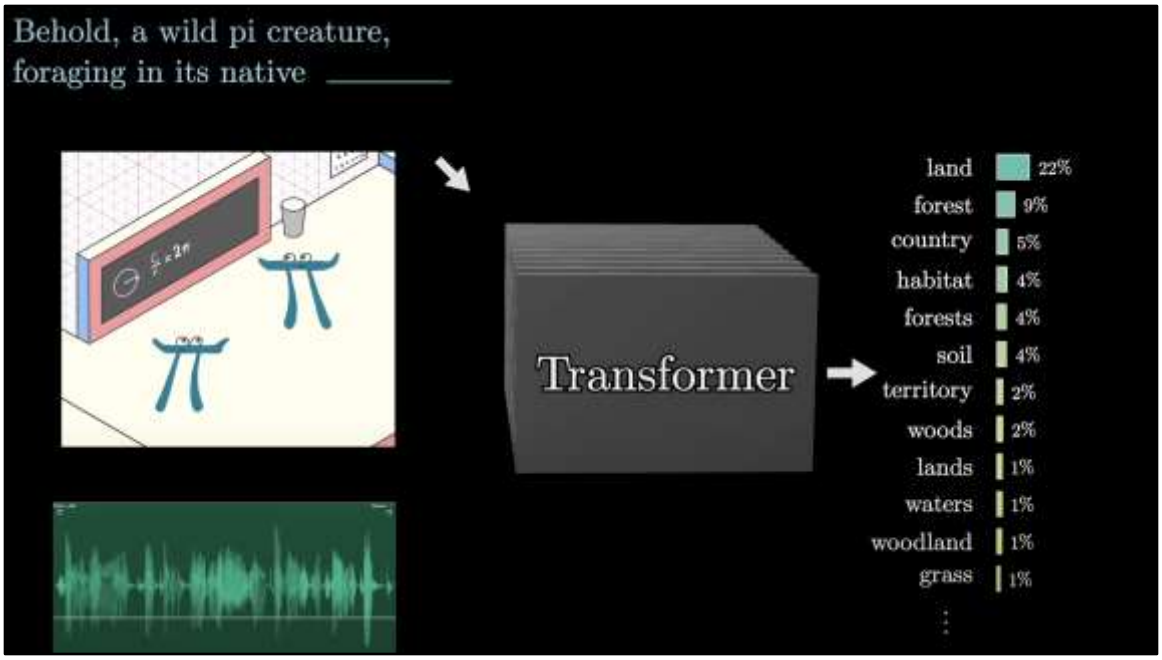
As we discussed in class, the embeddings are learned from data and depend on the corpus and the specific architecture of the transformer (or other ML method used to learn the embedding). Often they can reveal controversial ideas that exist in human language.

Generating Text: The Basics

- Generating Text: The Basics
- Task: Predict the **next token** based on context
- **Softmax**: Converts outputs into **probabilities**
  - Sigmoid : Step Function :: Softmax : Max
  - Forces all values to be between 0.0 and 1.0
  - Forces the values to add up to 1.0
- **Temperature**: Adjusts "creativity" in predictions
  - Low temperature → Predictable
  - High temperature → Surprising

When generating text, the model's primary task is to predict the **next token** based on the tokens it has already seen. This process is probabilistic, meaning the model doesn't just choose the most likely word—it assigns **probabilities** to all potential next tokens. These probabilities are calculated using a function called **softmax**, which ensures they sum up to 100%.

We can also control how **creative** the model's responses are by adjusting a parameter called **temperature**. At **low temperatures**, the model becomes more predictable, sticking to the most likely next token. At **high temperatures**, it explores less likely options, which can lead to more surprising or creative outputs. For example, at a low temperature, "The dog chased the..." might reliably predict "ball," while a high temperature might occasionally generate "neighbor" or even "squirrel."



Source: From today's assigned reading: https://www.3blue1brown.com/lessons/gpt

Controlling Text Generation

- Top-k sampling: Limits predictions to the **k most probable tokens**
- Top-p (nucleus) sampling: Focuses on a **dynamic probability threshold**
- Temperature, Top-k, and Top-p shape creativity and coherence
- Applications: Chatbots, story generation, creative writing

OpenAI models rely on **sampling-based methods** to generate text, ensuring outputs are natural and varied. One method is **top-k sampling**, where the model restricts its choices to the **k most probable tokens**. Another is **top-p sampling**, or **nucleus sampling**, which focuses on the smallest set of tokens whose combined probabilities exceed a certain threshold.

These parameters work together with **temperature** to balance creativity and coherence. For example, in chatbots, you might use a lower temperature and nucleus sampling for clear, concise answers. In creative writing, higher temperatures and more open sampling can produce more imaginative results. These controls are key to tailoring model behavior for different applications.

Challenges with Long-Range Dependencies

- Older models: Struggled with distant relationships

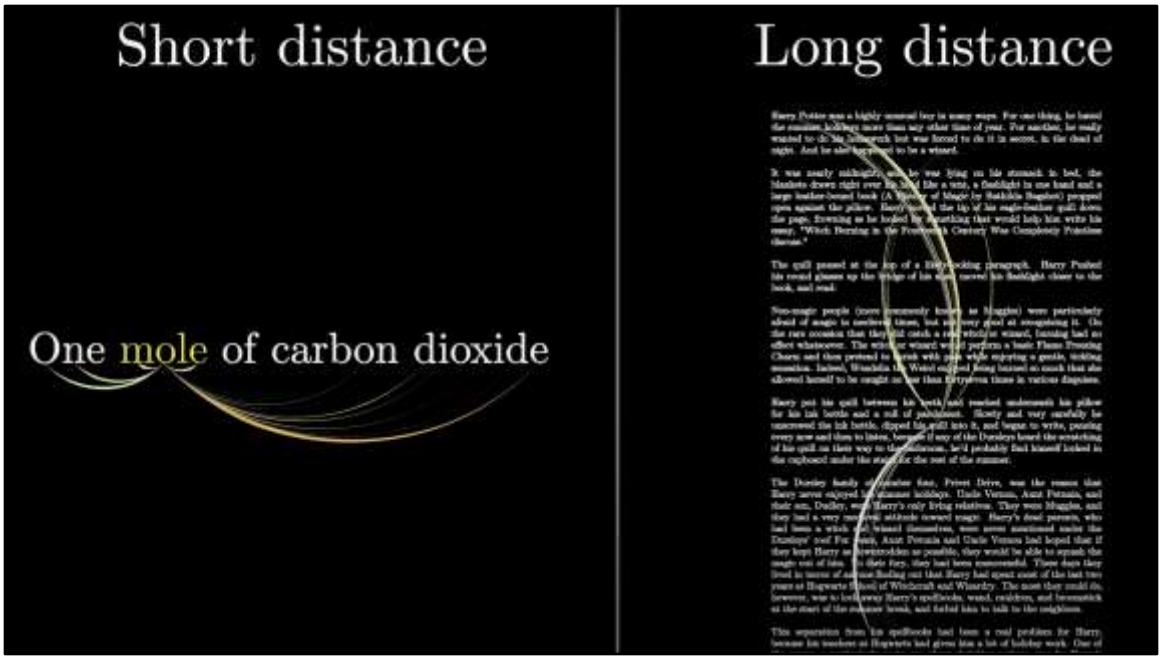
"The cat ate its food because **it** was hungry."

- Ambiguity: Does **it** refer **food**? **cat**? A **dog** mentioned earlier?
- Limited context: Older models had small **context windows**

One of the biggest challenges in language modeling is handling **long-range dependencies**—relationships between words or ideas that are far apart in a sentence or passage. Older models, like early neural networks or Markov chains, struggled to maintain these connections.

For example, consider the sentence: “The cat ate its food because it was hungry.” The word “it” is ambiguous—does it refer to “the cat” or “the food”? Resolving this depends on understanding the broader context of the sentence.

These models also had very small **context windows**, meaning they could only consider a fixed number of tokens at a time. This limitation made it hard to process complex sentences or longer passages, leading to errors or incoherence.



Source: https://www.3blue1brown.com/lessons/attention

How Transformers Address Context

- Attention mechanism: Weighs **relevant parts of input** text
- Example: “The cat ate its food because it was hungry.”
- Self-attention: Focuses on “cat” for **it was hungry**
- Large context windows:
  Modern models can process **thousands of tokens**

Transformers solve the problem of long-range dependencies with the **attention mechanism**, which allows them to focus on the most **relevant parts** of the input text, no matter how far apart they are.

For example, in the sentence “The cat ate its food because it was hungry,” a transformer uses **self-attention** to weigh the relationship between “it” and “the cat.” This helps the model resolve ambiguity in ways older methods couldn’t.

Another key innovation is the use of **large context windows**. Unlike earlier models that were limited to a small number of tokens, transformers can process thousands of tokens at once, allowing them to understand and generate coherent responses for much longer inputs. This is one of the reasons why transformers like GPT excel at tasks involving detailed or extended text.

Transformer Architecture Overview

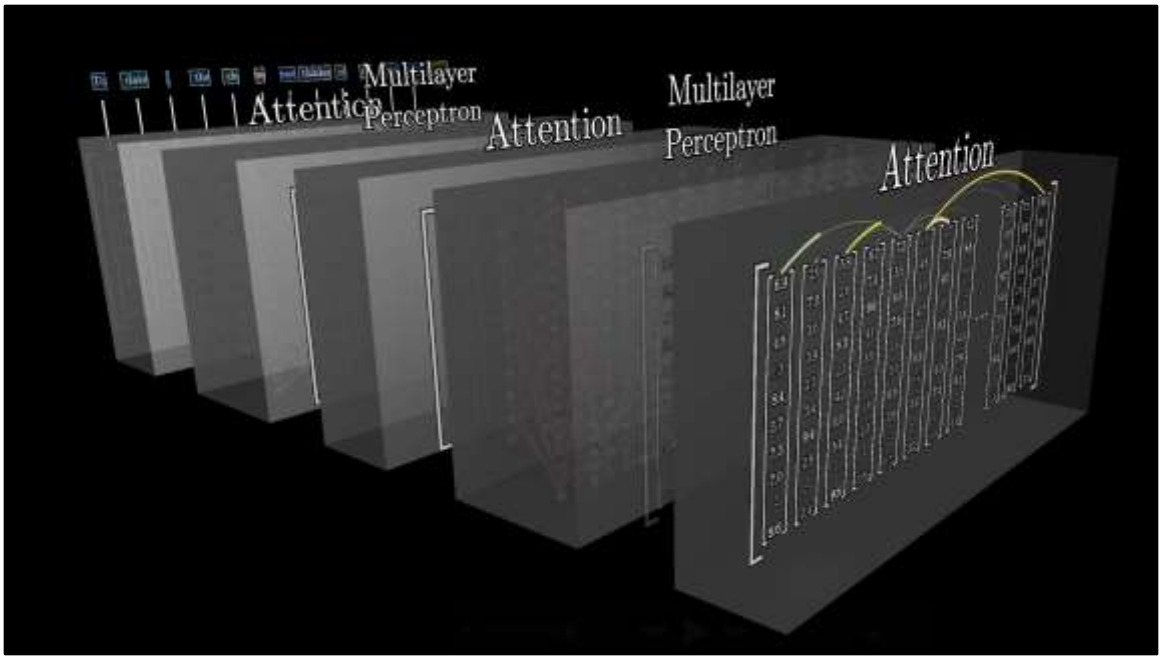
- Stacked layers
  Combine **attention** and **feed-forward** components
- Residual connections
  Preserve information across layers
- Layer normalization
  Stabilizes learning for better performance
- Why stacking matters
  Deeper models learn more complex patterns

At its core, the **transformer architecture** is a stack of layers, each combining two main components: **attention mechanisms** and **feed-forward neural networks**. Think of these layers as filters that process and refine the input text, gradually building a deeper understanding of it.

Transformers also use **residual connections**, which ensure that information flows smoothly through the layers without being lost or distorted. These connections allow the model to learn effectively, even when stacked to great depths.

To stabilize learning, transformers use **layer normalization**, which keeps the model's outputs consistent and prevents extreme values. This makes training faster and more reliable.

Finally, stacking layers matters because each additional layer helps the model learn increasingly complex patterns. For instance, the first few layers might identify simple relationships like word meanings, while deeper layers capture abstract ideas like tone or context.



Source: From the assigned readings:
https://www.3blue1brown.com/lessons/attention

Key Components of Transformer Layers

- Attention mechanism
  - Focuses on **relevant tokens** in input
- Feed-forward network
  - Multilayer perceptron (MLP)
  - Processes information **independently** for each token
- Residual connections
  - Prevent information loss during processing
- Layer normalization
  - Keeps outputs **stable and efficient**

Let's break down what happens in each **transformer layer**. The first key component is the **attention mechanism**, which determines which tokens in the input are most relevant to each other. This helps the model prioritize important relationships, like which word "it" refers to in a sentence.

The **feed-forward network** then processes the information for each token **independently**, applying learned transformations to enhance the model's understanding.

To ensure nothing important is lost as information passes through the layers, **residual connections** add the original input back into the layer's output. This makes it easier for the model to learn without overwriting earlier information.

Finally, **layer normalization** ensures that the outputs remain stable and consistent, making training more efficient and reducing the risk of errors.

Attention Mechanism: The Basics

- Key idea: Focus on **important words** in the input
- Attention scores: Measure **relevance** between tokens
- Example: “The cat ate because it was hungry.”
- Focus: Link “it” to “cat,” not “ate”

The **attention mechanism** is like a spotlight that helps the model focus on the most important parts of the input. Instead of treating all tokens equally, attention assigns a **score** to each token, measuring its relevance to the current task.

For example, in the sentence, “The cat ate because it was hungry,” the model needs to figure out that “it” refers to “cat.” The **attention mechanism** calculates relationships between tokens and assigns higher scores to more relevant ones. This ensures the model focuses on the right connections when making predictions.

How Attention Works: Queries, Keys, Values

Each token creates:

- **Query**: What are we looking for?
- **Key**: What information do we have?
- **Value**: The actual information

Example: Linking "it" to "cat"

- Query: "it" → What is "it" referring to?
- Key: "cat" → How relevant is "cat" to "it"?
- Value: "cat" → The actual meaning to focus on

The attention mechanism operates using three components for every token: **queries**, **keys**, and **values**.

- A **query** asks, "What are we looking for?" For example, "it" generates a query to find its reference.
- A **key** asks, "What information do we have?" Each word, like "cat," generates a key that signals its relevance.
- A **value** is the actual information stored in the token. If "cat" is deemed relevant, its value is used in the output.

In practice, the model compares queries and keys to compute **attention scores**. These scores determine how much focus the model places on each token, ensuring the right relationships are prioritized.

Self-Attention: Key Feature of Transformers

- Self-attention: Each token attends to **all tokens**, including itself
- Importance: Captures **relationships across the sequence**
- Efficient: Parallel processing speeds up computation

Transformers take the attention mechanism further with **self-attention**, where each token attends to all other tokens in the sequence—including itself. This is crucial because it allows the model to capture relationships regardless of their position.

For example, in "The cat ate because it was hungry," self-attention ensures that "it" can connect to "cat," even if they're several words apart. Importantly, this process is done **in parallel**, which makes transformers much faster and more efficient than older models like RNNs.

Multi-Headed Attention: The Basics

- Why multiple heads?
  - Focus on **different aspects** of relationships in the text
  - Example: Syntax vs. semantics
- How does it work?
  - Split queries, keys, and values into multiple **heads**
  - Process each head independently
- Combine results for richer understanding

Multi-headed attention extends the basic attention mechanism by using **multiple "heads"** to focus on different relationships in the text at the same time. Each head captures a unique perspective.

For example, one head might focus on **syntax**—like the grammatical relationships between words—while another focuses on **semantics**, understanding the meaning. This parallel processing allows transformers to capture complex, nuanced relationships more effectively.

Here's how it works: the model splits the queries, keys, and values into multiple smaller sets, called **heads**. Each head processes its part of the data independently. The outputs are then combined and fed into the next layer, providing a **richer representation** of the input text.

Why Multi-Headed Attention Matters

- Handles complex relationships: Syntax, semantics, and context
- Enables parallel processing for efficiency
- Example: “The cat chased the mouse because it was hungry.”
  - Head 1: Links “it” to “cat” (semantic focus)
  - Head 2: Tracks subject-verb agreement (syntax focus)

Multi-headed attention is powerful because it can **focus on different aspects** of the text simultaneously. This makes transformers exceptionally good at handling complex relationships.

For example, in the sentence “The cat chased the mouse because it was hungry,” one head might link “it” to “cat,” focusing on **semantics**, while another head ensures subject-verb agreement, focusing on **syntax**. This parallel processing makes transformers both effective and efficient.

By dividing attention across multiple perspectives, multi-headed attention helps the model create a **deeper, more nuanced understanding** of the input.

How Multi-Headed Attention Works

- Split: Divide queries, keys, and values into smaller subsets
- Independent processing: Each head focuses on a unique aspect
- Combine: Merge results into a single output

To understand the mechanics of multi-headed attention, think of it as dividing the model's attention resources into smaller chunks. Queries, keys, and values are **split into subsets**, and each subset is processed by an independent head.

This division allows each head to focus on **different parts** of the input or relationships within the text. Once all heads have done their work, the outputs are **combined** into a single representation, which is passed to the next layer.

This process ensures that no single head becomes overwhelmed and that the model can capture a wide variety of relationships, from **word meanings** to **sentence structure**.

Positional Encoding

- Transformers process inputs in parallel
- Need to encode **order** of tokens
- Example: Adding positional embeddings

Transformers process tokens in parallel, which makes them efficient but creates a challenge: how do they understand the **order** of words? For example, "The cat chased the dog" is very different from "The dog chased the cat." To solve this, transformers add **positional encodings** to token embeddings. These encodings, which can be sinusoidal or learned, help the model retain information about word order while processing sequences.

Fine-Tuning: Adapting Pretrained Models

- Process: Customize a general model for a specific task
- Saves time and resources: Build on pre-trained knowledge
- Example: Fine-tune GPT for medical Q&A or legal analysis

Fine-tuning is how we take a **general-purpose model** like GPT and adapt it to perform well on a specific task. Instead of training from scratch, which would require immense resources, we **fine-tune** the model using a smaller, specialized dataset. This saves both **time and resources** while leveraging the pre-trained model's broad knowledge.

For example, a healthcare provider might fine-tune GPT on medical texts to create a system for answering **medical questions**. Similarly, a law firm could fine-tune it on legal documents for tasks like **contract analysis** or summarization.

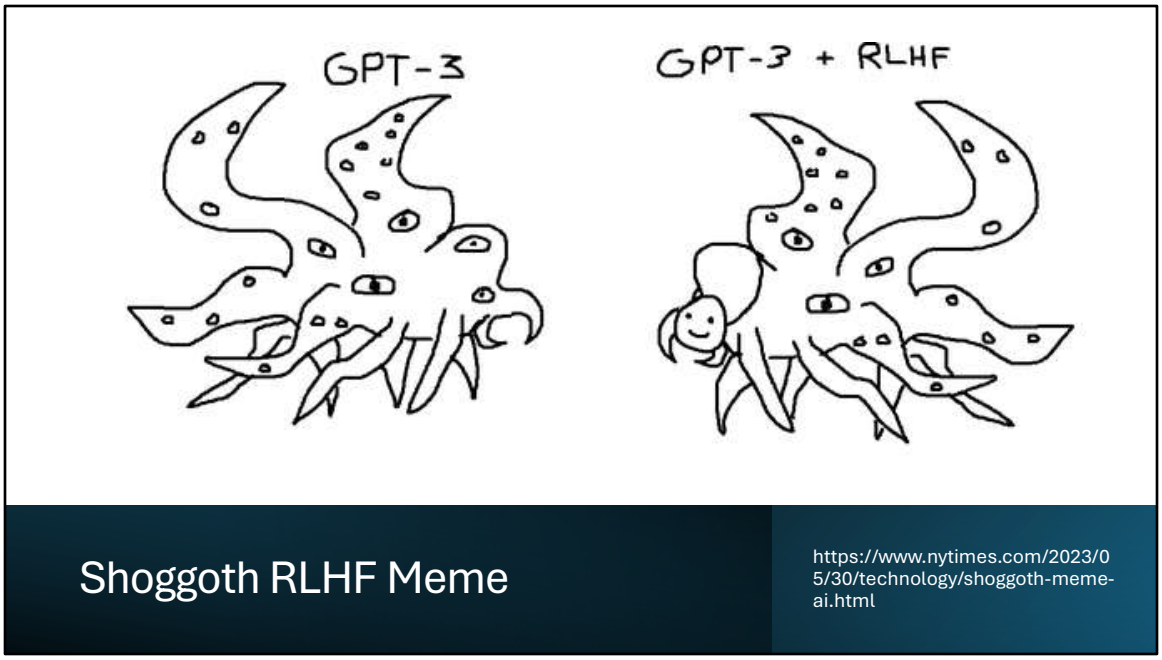
RLHF

Reinforcement Learning with Human Feedback

- Aligning Models with Human Intent
- Combines **human input** with reinforcement learning
- Human feedback guides the model's behavior
- Example: Training ChatGPT to provide polite, helpful answers

Reinforcment Learning with Human Feedback, or RLHF, is a method to align models with **human intent and values**. It involves using **human feedback** to guide the model's responses, ensuring they are polite, helpful, and aligned with the user's expectations.

For instance, when training ChatGPT, **human reviewers** rate the model's responses during its fine-tuning phase.
These ratings are then used in a reinforcement learning loop to adjust the model's behavior, prioritizing answers that align better with user needs. RLHF is one of the reasons ChatGPT excels at producing **user-friendly and safe interactions**.



[I mentioned this in class. The meme is shown above.]

This slide shows the Shoggoth meme, which has become a popular metaphor in the AI community for explaining the inner workings and quirks of large language models (LLMs). The image—a monstrous creature with a smiley-face mask—represents the disconnect between the polished, user-friendly interface we interact with and the complex, opaque, and often unpredictable processes beneath the surface. The smiley face, labeled "RLHF" (Reinforcement Learning from Human Feedback), symbolizes the fine-tuning process used to align AI outputs with human values, making the system appear more coherent and helpful. The meme gained prominence as chatbots like Bing and ChatGPT became mainstream, especially when their unintended or bizarre behaviors, such as Bing's infamous meltdown, raised questions about their reliability and safety. It humorously encapsulates the tension between the powerful, alien nature of these systems and the efforts to make them appear safe and trustworthy.

Source: https://www.nytimes.com/2023/05/30/technology/shoggoth-meme-ai.html

Applications

- Chatbots (e.g., customer service, tutoring)
- Translation (e.g., real-time, multilingual systems)
- Creative writing (e.g., poetry, screenwriting, brainstorming)

Transformers power some of the most impressive AI applications today.

For instance, **chatbots** like ChatGPT excel at customer service, personal tutoring, and interactive storytelling.

Translation systems now handle real-time, multilingual communication, bridging language barriers.

Creative fields could also benefit, with AI assisting in **poetry**, **screenwriting**, and even generating ideas.

Limitations and Concerns

- **Context window**: Limited capacity for long documents
- **Hallucinations**: Generates false or misleading information
- **Biases**: Reflects biases in training data
- **Ethical concerns**: Energy consumption, misinformation risks

However, these models have significant **limitations**. As we've already discussed, they are constrained by their **context window**, meaning they may lose coherence in longer documents.

In addition, they can also produce **hallucinations**, generating false or misleading information confidently. **Biases** in the training data can lead to problematic or unfair outputs, raising important ethical questions. Lastly, the **energy consumption** of training and running these models is immense, contributing to environmental concerns. As transformers become more widely used, the risks of **misinformation** and misuse, such as deepfakes or malicious content, are critical to address. This underscores the importance of ethical development and responsible use. We will explore these issues—and others—in the next unit.

Wrap-Up and Next Steps

- Transformers revolutionized AI with **attention**
- GPT is a **powerful generative model**
- Next time: **LLMs in agents** and **RAG**

To wrap up, we've seen how **transformers** revolutionized AI with the concept of **attention**, enabling models like GPT to process language in unprecedented ways. GPT's ability to generate text, adapt to tasks, and model long-range dependencies is truly transformative. In our next lecture, we'll explore how large language models are embedded in **agents** and how **Retrieval-Augmented Generation (RAG)** addresses the context window limitation by incorporating external knowledge sources.